CS114: Fundamentals of Programming

Class: BECE2018

Lab 10: Functions – II

Instructor: Qurrat-ul-ain Babar

Lab 10: Functions – II

The aim of this lab is to introduce some other basics of Functions like default parameters in functions.

Our aims today are:

- Default parameters in functions
- Variables Scope

Deliverables

1. Perform all **Tasks**. Write the code and take screen shots of the output of task and share both in a word document. The code should be readable and not a screen shot.

Upload single word document on LMS. Follow the naming convention for file. No late submissions will be accepted.

Deadline: Check submission deadline on LMS.

Default Parameters in Functions

A default argument is a value provided in function declaration that is automatically assigned by the compiler if caller of the function doesn't provide a value for the argument with default value.

Following is a simple C++ example to demonstrate use of default arguments. We don't have to write 3 sum functions, only one function works by using default values for 3rd and 4th arguments.

Example 1

```
#include <iostream>
using namespace std;

// A function with default arguments, it can be called with
// 2 arguments or 3 arguments or 4 arguments.
int sum(int x, int y, int z=0, int w=0)
{
    int r;
    r = (x + y + z + w);
    return r;
}

/* Drier program to test above function*/
int main()
{
    cout << sum(10, 15) << endl;
    cout << sum(10, 15, 25) << endl;
    cout << sum(10, 15, 25, 30) << endl;

    return 0;
}
```

Example 2

```
#include <iostream>
using namespace std;

int showVolume(int length, int width = 1, int height = 1);

int main()
{
    int vola,volb,volc;
    vola = showVolume(4, 6, 2);
    volb = showVolume(4, 6);
    volc = showVolume(4);

    return 0;
}

int showVolume(int length, int width=1, int height=1)
{
    int volume;
    volume=length*width*height;
    return volume;
}
```

Variables Scope

All the variables have their area of functioning, and out of that boundary they don't hold their value, this boundary is called scope of the variable. For most of the cases it's between the curly braces, in which variable is declared that a variable exists, not outside it. We will study the storage classes later, but as of now, we can broadly divide variables into two main types,

- Global Variables
- Local variables

Global Variables

Global variables are those, which are once declared and can be used throughout the lifetime of the program by any class or any function. They must be declared outside the main () function. If only declared, they can be assigned different values at different time in program lifetime. But even if they are declared and initialized at the same time outside the main () function, then also they can be assigned any value at any point in the program.

Example 3

```
#include <iostream>
using namespace std;

int x;                  // Global variable declared

int main()
{
    x=10;                   // Initialized once
    cout <<"first value of x = "<< x;
    x=20;                   // Initialized again
    cout <<"Initialized again with value = "<< x;

    return 0;
}
```

Local Variables

Local variables are the variables which exist only between the curly braces, in which its declared. Outside that they are unavailable and leads to compile time error.

Example 4

```
#include <iostream>
using namespace std;

int main()
{
    int i=10;

    if(i < 20)        // if condition scope starts
    {
        int n=100;    // Local variable declared and initialized
    }                // if condition scope ends
    cout << n;       // Compile time error, n not available here

    return 0;
}
```

Task 1:

Write a program to find area of a triangle by using default values and values passed to the function. The formula for triangle is ½ * base * triangle.

Task 2:

Add comments in the following program to show the scope of the variables and mention if any error is going to occur.

```
#include <iostream>
using namespace std;

int weight;

int main()
{
    weight =  3;
    cout  << "Weight is  " << weight  << endl;

    return 0;
}
```

Task 3:

Add comments in the following program to show the scope of the variables and mention if any error is going to occur.

```
#include <iostream>
using namespace std;

int weight;

int main()
{
    {
        double price  = 2.0,
        height  = 3.1,
        length  = 2.4;
        cout << "Price is " << price << endl;
        cout << "Length is " << length << endl;
        cout << "Height is " << height  << endl;
    }

    weight = 3;

    return 0;
}
```

Task 4:

Add comments in the following program to show the scope of the variables and mention if any error is going to occur.

```
#include <iostream>
using namespace std;

int weight;

int main()
{
    {
        double price = 2.0,
        height = 3.1,
        length = 2.4;
        cout << "Price is " << price << endl;
        cout << "Length is " << length << endl;
        cout << "Height is " << height << endl;
    }
    weight = 3;
    cout << "Price is " << price << endl;
    cout << "Length is " << length<< endl;
    cout << "Height is " << height<< endl;

    return 0;
}
```